Claude Code in Cursor

A beginner's step-by-step study guide

Project A 00 · Researched from official docs on 6 Nov 2026 · Windows 11

How to use this guide: work through the lessons in order. Each lesson ends with a short checklist you can tick off. Don't rush; it's fine to do one lesson per day. Commands to type are shown in `grey boxes`.

Contents

1. Lesson 1: Key ideas (what is what)
2. Lesson 2: Install everything
3. Lesson 3: Connect Claude Code to Cursor
4. Lesson 4: Organize your project folder
5. Lesson 5: Talk to Claude: prompts, @files, modes
6. Lesson 6: The slash commands you'll use most
7. Lesson 7: Review changes and undo mistakes
8. Lesson 8: Create your own subagents
9. Lesson 9: Create your own /commands (skills)
10. Lesson 10: A daily workflow you can repeat
11. Cheat sheet, practice exercises and resources

Things changed recently. Many older videos and blogs are out of date:

- The `/agents` wizard was removed. You now create agents as files, or ask Claude to (Lesson 8).
- "Skills" are now the recommended way to make custom `/commands` (Lesson 9).
- The `#` shortcut for saving memory seems to be gone. Use `/memory` instead.
- New sessions may start in **auto mode**, so Claude can work without asking before each step. Always check the mode shown at the bottom of the screen.

Lesson 1: Key ideas (what is what)

Term	Meaning in plain words
Cursor	A code editor (IDE) based on VS Code. It's where you see and edit your files.
Claude Code	Anthropic's AI coding assistant. It can read your project, edit files and run commands. You can use it in the **terminal** (CLI) or in a **chat panel** (the extension).
Terminal / CLI	The text window at the bottom of Cursor (open it with `Ctrl+``). You type `claude` there to start.
Prompt	The message you type to Claude.
Slash command	A command that starts with `/`, like `/clear`. It controls Claude Code itself.
CLAUDE.md	A text file of notes about your project. Claude reads it at the start of every session, like a briefing.
Subagent	A helper Claude with its own job and tools (e.g. "researcher"). The main Claude hands it a task and gets back a report.
Skill	A saved set of instructions you run as your own `/command`.
Diff	A side-by-side view showing what changed in a file: red is removed, green is added.
Git / commit	Git saves snapshots ("commits") of your project, so you can always go back.

- ☐ I can explain the difference between Cursor and Claude Code.
- ☐ I know what CLAUDE.md is for.

Lesson 2: Install everything

1. **Check your plan.** Claude Code needs a paid Claude plan (Pro, Max, Team or Enterprise) or a Claude Console account. The free plan doesn't include it.
2. **Install the Claude Code CLI.** Open PowerShell and run:

   ```
   irm https://claude.ai/install.ps1 | iex
   ```

   Close the window, open a new one, and check that it worked:

   ```
   claude --version
   ```

   If something looks wrong, `claude doctor` checks your installation.
3. **Install Git for Windows** (recommended) from git-scm.com. It lets Claude use Bash and lets you save versions of your work.

4. **Install the extension in Cursor.** Press `Ctrl+Shift+X`, search **Claude Code** (by Anthropic, id `anthropic.claude-code`) and click **Install**. If it doesn't appear afterwards, press `Ctrl+Shift+P` and run **Developer: Reload Window**.
5. **Sign in.** In Cursor's terminal, type `claude` and follow the browser login.

> **Note:** the extension and the CLI are two separate installs. The extension does *not* add the `claude` command to your terminal, so you need step 2 as well.

- ☐ `claude --version` prints a version number.
- ☐ The Claude Code extension shows in Cursor's Extensions list.
- ☐ I'm logged in.

Lesson 3: Connect Claude Code to Cursor

You can use Claude in two places. Both share the same conversation history.

Option A: Terminal (CLI)

1. In Cursor, open your project folder (**File** → **Open Folder** → `Project A 00`).
2. Open the terminal with `Ctrl+`` and type `claude`.
3. Type `/ide` to check it's connected to Cursor. When connected, Claude can see the file you have open and any text you've selected, and its edits show up as diffs in the editor.

Option B: Chat panel

Click the **Spark icon** (✻) at the top right of an open file, or **"✻ Claude Code"** in the bottom status bar. Press `Ctrl+Shift+Esc` to open it in a new tab.

> **Troubleshooting:** if `/ide` says "No available IDEs detected", the extension isn't installed. If it says "WebSocket is not open", reload the Cursor window (`Ctrl+Shift+P` → Developer: Reload Window) and run `/ide` again.

- ☐ I started `claude` in Cursor's terminal and `/ide` shows it's connected.
- ☐ I opened the chat panel once.

Lesson 4: Organize your project folder

This is the recommended layout. You don't need every file on day one; add them as you need them.

```
Project A 00/
├─ CLAUDE.md                  project notes, read every session     (commit)
├─ CLAUDE.local.md            your private notes                      (don't commit)
├─ README.md                  explains the project to humans         (commit)
├─ .gitignore                 files git should ignore                (commit)
├─ .mcp.json                  connections to outside tools (MCP)     (commit)
├─ docs/                      guides like this one                   (commit)
└─ .claude/
   ├─ settings.json           team permissions and settings          (commit)
   ├─ settings.local.json     your personal settings                 (don't commit)
   ├─ agents/                 your subagents, one .md file each      (commit)
   ├─ skills/<name>/SKILL.md  your own /commands                      (commit)
   └─ rules/                  extra instruction files by topic       (commit)
```

CLAUDE.md: your project's briefing

- Keep it short (under about 200 lines): what the project is, how to build and test it, and rules to follow.
- Once you have code, run `/init` and Claude writes a starter CLAUDE.md for you.
- To edit it later, run `/memory` or just ask Claude: "add to CLAUDE.md that we use 2-space indents".
- It can include other files with `@README.md`.

.gitignore: keep personal files private

```
.claude/settings.local.json
CLAUDE.local.md
```

Settings example (.claude/settings.json)

This one lets Claude run tests without asking and stops it reading your secret `.env` files:

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": ["Bash(npm run test *)"],
    "deny": ["Read(./.env)", "Read(./.env.*)"]
  }
}
```

The file must be strict JSON: no comments and no trailing commas. Run `/status` to see which settings files were loaded.

Cursor rules vs CLAUDE.md

- Cursor's own AI reads `.cursor/rules/*.mdc` and `AGENTS.md`. Claude Code reads `CLAUDE.md`. They are separate.
- **To share one set of rules between both:** write them in `AGENTS.md`, then make the first line of `CLAUDE.md` `@AGENTS.md`.
- `/import cursor --dry-run` previews copying your Cursor rules, MCP servers, commands and agents into Claude Code.

☐ My project has a CLAUDE.md and a .gitignore.

☐ I know which files to commit and which to keep private.

Lesson 5: Talk to Claude: prompts, @files, modes

Writing good prompts

- **Be specific:** "Add a button to `@src/App.tsx` that resets the form" beats "make it better".
- **Give Claude a way to check its work:** "...then run the tests and fix any failures".
- **Ask questions first** when you're new to a codebase: "Give me an overview of this project."

Special keys in the prompt

Type this	What it does
`@filename`	Attaches a file. `@folder/` attaches a folder. Press Tab to autocomplete.
`!command`	Runs a shell command, e.g. `!git status`. Claude sees the output.
`/`	Opens the slash command menu.
`?`	Shows keyboard shortcut help.
`Shift+Enter`	New line without sending. Run `/terminal-setup` if it doesn't work.
`Alt+V`	Pastes an image (screenshot) on Windows.

Modes: how much Claude can do without asking

Press `Shift+Tab` to cycle modes (on Windows, `Alt+M` if Shift+Tab doesn't work). The current mode shows at the bottom of the screen.

Mode	Behaviour	Use it when
Manual	Asks before every edit and command.	You're learning: **recommended to start**
Accept edits	Edits files without asking, still asks for commands.	You trust the plan
Plan	Only reads and plans; changes nothing until you approve.	Bigger tasks

Auto	Works without asking; a safety check blocks risky actions.	Once you're comfortable

Stopping Claude

- `Esc`: stops Claude now and keeps what it did so far. Then tell it what to do instead.
- `Esc Esc` (on an empty prompt): opens the rewind menu (Lesson 7).
- `Ctrl+C` twice: quits Claude Code.

Chat panel shortcuts

- `Ctrl+Esc`: switches focus between the editor and Claude.
- `Alt+K`: inserts a reference to the selected lines, e.g. `@app.js#5-10`.

These may clash with Cursor's own AI shortcuts. If one doesn't work, check Cursor's keyboard settings.

☐ I attached a file with `@`.

☐ I switched modes with Shift+Tab and found the mode on screen.

☐ I stopped Claude with Esc.

Lesson 6: The slash commands you'll use most

★ marks the ones to learn first. Type `/help` any time for the full list.

Command	What it does
★ `/init`	Creates a starter CLAUDE.md by reading your code.
★ `/ide`	Checks or fixes the connection to Cursor.
★ `/clear`	Starts a fresh conversation. Use it between unrelated tasks.
★ `/plan [task]`	Plan mode: Claude plans first and doesn't edit until you approve.
★ `/diff`	Shows all uncommitted changes, including Claude's.
★ `/code-review`	Checks your current changes for bugs. `--fix` applies the fixes. `/review` also works.
★ `/rewind`	Rolls code and conversation back to an earlier point.
★ `/resume`	Reopens an earlier conversation.
`/compact`	Summarizes a long conversation to free up space.
`/context`	Shows what's taking up Claude's memory.
`/model`	Switches the AI model (also `Alt+P`).
`/memory`	Edits CLAUDE.md and memory settings.
`/permissions`	Controls what Claude may do without asking.
`/simplify`	Tidies up changed code (doesn't look for bugs).
`/security-review`	Checks your changes for security problems.
`/status`, `/usage`	Version, account and loaded settings; your usage and limits.
`/config`	Settings screen (terminal only).
`/doctor`	Checks and fixes setup problems.
`/powerup`	Interactive lessons inside Claude Code. **Great for beginners.**

Commands change between versions. If one isn't there, check `/help` and `claude --version`.

- ☐ I ran `/help`, `/clear` and `/diff`.
- ☐ I tried `/powerup`.

Lesson 7: Review changes and undo mistakes

Reviewing in Cursor

- In **Manual** mode, each edit opens a side-by-side diff. Click **Accept** or **Reject**, or type what Claude should do instead.
- You can change the code inside the diff yourself; Claude is told what you changed.
- Before committing, run `/diff` to see everything, then `/code-review`.

Undoing

- `/rewind` (or `Esc Esc`): pick a point and go back, for the code, the conversation, or both.
- In the chat panel, each message has a rewind button: "Rewind code to here".
- **Git is your safety net:** commit when something works. Then you can always return to that point.

☐ I accepted one change and rejected another.

☐ I used `/rewind` once.

Lesson 8: Create your own subagents

A subagent is a helper with one job. Your project already has one: `.claude/agents/researcher.md`.

Where they live

- **This project only:** `.claude/agents/name.md` (commit it).
- **All your projects:** `C:\Users\<you>\.claude\agents\name.md`

The easy way: ask Claude

```
Create a project subagent in .claude/agents/ called code-reviewer that
reviews my changes for bugs. Read-only tools only. Use the sonnet model.
```

The manual way: write the file

```
---
name: code-reviewer
description: Reviews recent code changes for bugs and readability. Use proactively after edits.
tools: Read, Glob, Grep
model: sonnet
---
You are a careful code reviewer. Point out bugs, unclear names, and
missing error handling, with file:line references and suggested fixes.
```

Field	Meaning
`name`	Required. The agent's name.
`description`	Required. Tells Claude *when* to use it. "Use proactively" makes Claude use it more often.
`tools`	Optional. Which tools it may use. Leave it out to allow all tools.
`model`	Optional. `sonnet`, `opus`, `haiku`, `fable`, or `inherit`.
Text below `---`	The agent's instructions.

Using it

- Just ask: "Use the code-reviewer subagent to check my changes."
- Or type `@agent-code-reviewer`.
- Claude also picks one automatically when your request matches its description.

> **Restart once** if the `.claude/agents` folder didn't exist when you started `claude`. After that, new agents are picked up within a few seconds.

☐ I used the researcher agent once.

☐ I created a second agent myself.

Lesson 9: Create your own /commands (skills)

A skill is a saved prompt you run as `/name`. Create `.claude/skills/explain/SKILL.md`:

```
---
name: explain
description: Explain a file or piece of code in beginner-friendly terms.
disable-model-invocation: true
---
Explain $ARGUMENTS to a beginner. Cover what it does, how it works step
by step, and one thing I could try changing to learn more.
```

Now type `/explain @src/app.js`. `$ARGUMENTS` is replaced by what you type after the command. `disable-model-invocation: true` means only you can run it, not Claude on its own.

If a new skill doesn't show up, run `/reload-skills`. `/skills` lists them all. Older files in `.claude/commands/name.md` still work too.

- ☐ I made a skill and ran it with `/`.

Lesson 10: A daily workflow you can repeat

1. **Explore.** "Give me an overview of this project" or "find the files that handle login". For big questions: "use a subagent to investigate..."
2. **Plan.** Switch to plan mode (`Shift+Tab` or `/plan`). Read the plan. Ask questions. Approve it when it looks right.
3. **Code.** Let Claude make the changes. Tell it how to check its work: "run the tests and fix failures".
4. **Review.** `/diff`, read the changes in Cursor, then `/code-review`.
5. **Commit.** "Commit this with a descriptive message."
6. **Reset.** `/clear` before the next unrelated task.

> **Golden rule:** press `Esc` as soon as Claude goes the wrong way. If you've corrected the same thing twice and it's still wrong, run `/clear` and write a clearer prompt instead of correcting a third time.

Cheat sheet

Keys / command	Action	Keys / command	Action
`claude`	Start	`Esc`	Stop Claude
`claude --continue`	Continue last chat	`Esc Esc`	Rewind menu
`Shift+Tab`	Change mode	`@file`	Attach file
`/clear`	Fresh start	`!cmd`	Run shell command
`/plan`	Plan first	`Ctrl+R`	Search past prompts
`/diff`	See changes	`Ctrl+O`	Show full detail
`/code-review`	Check for bugs	`Ctrl+Esc`	Editor ↔ Claude panel
`/rewind`	Undo	`Alt+K`	Reference selection
`/help`	All commands	`?`	Shortcut help

Practice exercises

1. In this project, ask Claude: "Create a simple `hello.html` page with a button that shows the current time." Review the diff, then accept it.
2. Ask Claude to change the button colour. Then use `/rewind` to undo it.
3. Run `/init` and read the CLAUDE.md it writes.
4. Use plan mode to plan "add a dark mode toggle", read the plan, then approve it.
5. Run `/code-review` on your changes, then commit them.

6. Create the `/explain` skill from Lesson 9 and use it on `hello.html`.

Resources

- Claude Code in VS Code and Cursor: code.claude.com/docs/en/vs-code
- Setup: code.claude.com/docs/en/setup
- Quickstart: code.claude.com/docs/en/quickstart
- Best practices: code.claude.com/docs/en/best-practices
- Common workflows: code.claude.com/docs/en/common-workflows
- Slash commands: code.claude.com/docs/en/commands
- Keyboard shortcuts and modes: code.claude.com/docs/en/interactive-mode
- Memory and CLAUDE.md: code.claude.com/docs/en/memory
- Settings: code.claude.com/docs/en/settings
- Subagents: code.claude.com/docs/en/sub-agents
- Skills: code.claude.com/docs/en/skills
- Cursor rules: cursor.com/docs/context/rules
- Free course, "Claude Code in Action": anthropic.skilljar.com/claude-code-in-action

Not fully verified: the removal of the `#` memory shortcut, whether auto mode is the default on your plan, and Cursor-specific shortcut clashes. Claude Code updates often, so check `claude --version` and `/help` if something differs.